

# GestorPro

Aplicação web para controle de pedidos, clientes e estoque

---

## Integrantes do Grupo

- Breno Tales de Oliveira Leite - 2840482423029
- Daniel Fredi Soares Pereira - 2840482421054

# GestorPro

Regras, objetivo e funcionalidades

## Sobre o sistema:

- Sistema web desenvolvido em Python com Flask.
- Permite cadastrar clientes.
- Permite cadastrar materiais e controlar estoque por bobinas.
- Permite criar pedidos informando cliente, material, largura, altura e quantidade.

## Primeira etapa do projeto:

- Definimos o problema: controlar pedidos, clientes e estoque de materiais.
- Modelamos as principais entidades do sistema: Cliente, Pedido, Material, CategoriaMaterial e BobinaEstoque.
- Criamos a estrutura inicial do projeto em Python com Flask.
- Implementamos o banco de dados SQLite com Flask-SQLAlchemy.
- Criamos as primeiras telas para listar e cadastrar pedidos, clientes e materiais.
- Implementamos regras de negócio, como cálculo de consumo de material e controle de estoque por bobinas.
- Organizamos o código em camadas: models, controllers, services, templates e static.

# Arquitetura OO

Diagrama UML das principais classes e decisões de arquitetura.

## Conceitos OO Aplicados



### Encapsulamento

A herança aparece em todos os models, que herdam de db.Model. Com isso, nossas classes passam a ter comportamento de entidades persistidas no banco, permitindo consultas, cadastro, edição e exclusão.



### Herança

O encapsulamento aparece nas propriedades dos models e nas funções de serviço. Por exemplo, a classe Material possui propriedades como quantidade\_bobinas e metros\_disponiveis, então o restante do sistema não precisa saber como esse cálculo é feito internamente.



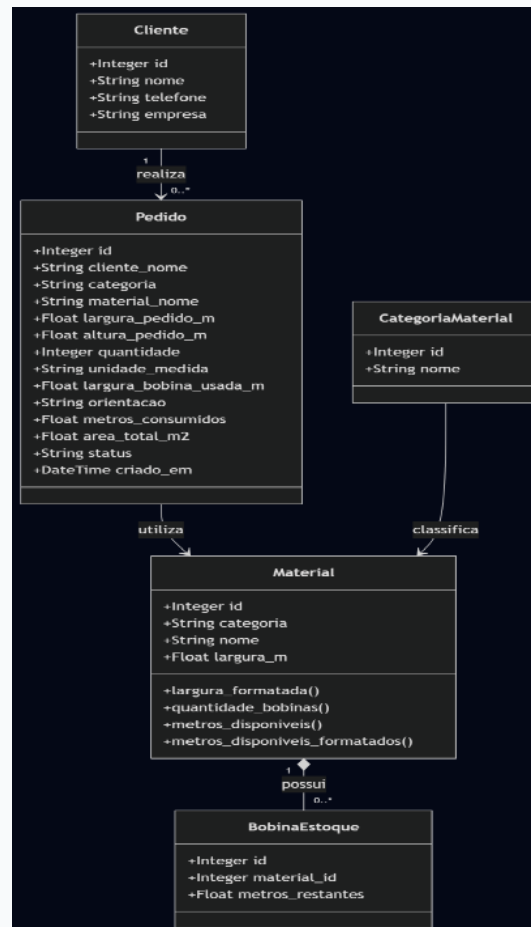
### Separação de responsabilidades

a arquitetura prioriza separação de responsabilidades. Os models representam os dados, os controllers recebem as requisições e direcionam as telas, os services concentram as regras de negócio, e os templates cuidam da interface visual.



### Interface

A interface com o usuário fica nos templates HTML, os controllers fazem a ponte entre a tela e a lógica, e os models representam os dados. Com isso temos Models para dados, controllers para controle das requisições e templates para visualização.



# Decisões de Design

O projeto foi separado em camadas:

## MODELS

Representam as tabelas do banco.

## CONTROLLERS

Recebem requisições, processam formulários e escolhem as telas.

## SERVICES

Concentram regras de Negócios.

## TEMPLATES

Telas HTML renderizadas com Jinja.

## STATIC

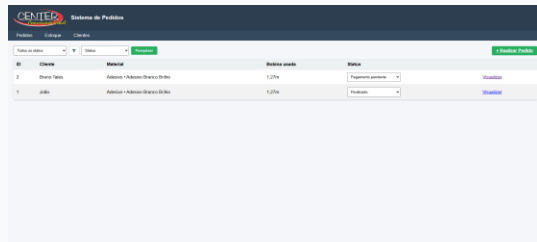
CSS, imagens e JavaScript

### Nossas decisões de design e por que as tomamos

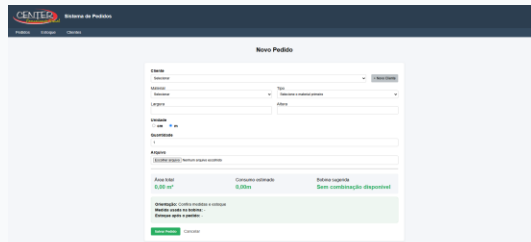
1. **Separação em camadas** — Dividimos o projeto em models, controllers, services, templates e static para não misturar banco de dados, regras de negócio e interface. Isso deixa o código mais organizado e mais fácil de manter.
2. **Uso de SQLite com Flask-SQLAlchemy** — Escolhemos SQLite por ser um banco simples, local e suficiente para o projeto acadêmico. Usamos Flask-SQLAlchemy para trabalhar com classes Python em vez de escrever SQL manualmente em todas as operações.
3. **Regras de negócio nos services** — Colocamos cálculos e operações importantes, como consumo de estoque, devolução de material e cálculo de melhor aproveitamento da bobina, dentro da camada services. Assim os controllers ficam mais limpos e a lógica pode ser reutilizada.
4. **Interface web com Flask, HTML, CSS e Jinja** — Escolhemos Flask porque permite criar uma aplicação web em Python de forma simples. O Jinja foi usado para preencher as páginas HTML com dados vindos do sistema, como pedidos, clientes e materiais.

# Telas Implementadas

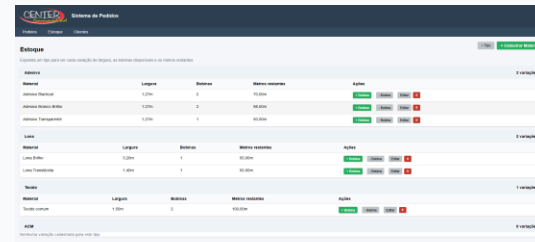
Capturas de tela de cada tela com breve legenda



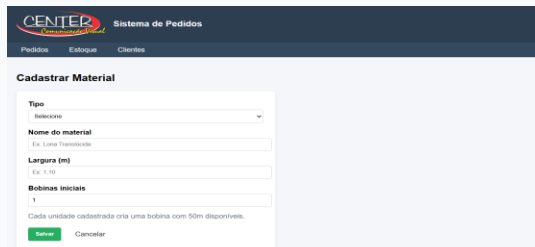
Tela de pedidos:



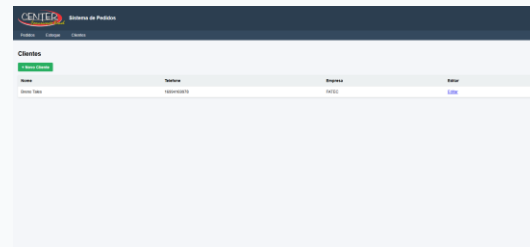
Tela de novo pedido:



Tela de estoque:



Tela de cadastro de material:



Tela de clientes:

# Demonstração ao Vivo

*Apresentação do sistema funcionando*

Cadastrar estoque -> Realizar Pedido -> Cadastrar Cliente -> Finalizar Pedido -> Progresso do Pedido -> Cancelar Pedido

## Desafios & Aprendizados

O que foi mais difícil? Como resolvemos? O que faríamos diferente?



### Maior Desafio

O maior desafio foi implementar o estoque por bobinas. O sistema não apenas cadastra pedidos, ele precisa calcular qual material oferece o melhor aproveitamento, verificar se há metragem suficiente e atualizar o estoque automaticamente. Além disso, quando um pedido é editado ou cancelado, a metragem precisa ser devolvida corretamente ao estoque. Isso exigiu separar bem a lógica nos services para não deixar os controllers muito complexos.



### Como Resolvemos

Para resolver esse desafio, separamos a lógica de estoque em arquivos de service. Em vez de deixar todo o cálculo dentro das rotas, criamos funções específicas para consumir material, devolver material, adicionar bobinas, remover bobinas e calcular o melhor aproveitamento. Isso deixou o código mais organizado e facilitou testar e corrigir a regra de negócio.



### Faríamos Diferente

Poderíamos ter criado testes automatizados para as regras de negócio, principalmente para o cálculo de aproveitamento das bobinas e para o consumo ou devolução ao estoque. Como essa é a parte mais importante e sensível do sistema, testes ajudariam a garantir que alterações futuras não quebrem esses cálculos.

# Conclusão

O que foi entregue, extras implementados e próximos passos



## O que entregamos

- O sistema permite gerenciar clientes, pedidos e estoque de materiais de forma integrada, com cadastro, edição, consulta e atualização de dados.
- Foram implementadas 8 telas principais, cobrindo as funcionalidades essenciais do sistema e a navegação entre as áreas de pedidos, clientes e estoque.
- O projeto foi organizado em camadas, com destaque para a service layer, onde ficam as principais regras de negócio, como cálculo de aproveitamento, consumo e devolução de estoque.
- Repositório do projeto: <https://github.com/brenotales1/sistema-pedidos.git>



## Próximos Passos

- Adicionar o cadastro de materiais via entrada de NF.
- Substituir banco de dados por algum mais robusto (como MySQL), devido possível concorrência e escalabilidade do projeto.
- Implementar uma API para notificar clientes o status do pedido automaticamente ao alterá-lo no sistema.
- O sistema foi desenvolvido inicialmente para fins acadêmicos, mas sua estrutura permite evolução para uma solução comercial voltada a empresas de comunicação visual, gráficas rápidas e negócios que trabalham com encomendas personalizadas.



# Obrigado!

**ProGestor**

Perguntas?

Grupo: Breno Tales, Daniel Fredi



*Repositório:* <https://github.com/brenotales1/sistema-pedidos>